

DOMAIN-DRIVEN DESIGN, TDD AND CLEAN ARCHITECTURE

GROUP COURSEWORK

Department of Networks
School of Computing and Informatics Technology, Makerere University

Instructor: Kasozi W. Vincent

Group size: 5 students | Total: 100 marks

1. What you are required to do

Choose one small application domain. Define six business rules for that domain. Use the rules to build a small domain model, organise it using Clean Architecture, and verify the important behaviour with tests. Your submission is a slide deck plus the small implementation used as evidence.

The application should be small. Do not build a full management system. Two connected use cases are enough. Use in-memory persistence. A web interface, production database, deployment, microservices and message brokers are not required.

AI tools may be used. If you use them, state where they were used and be ready to justify the design and implementation produced. You will not earn marks for work you cannot explain.

2. Choose the domain

Pick a domain that has clear rules and state changes. Examples include room booking, equipment borrowing, appointment scheduling, parcel collection, order processing, school term management or event registration.

On Slide 2 state: the problem, what is inside your scope, what is outside your scope, and the main terms used in the domain.

3. Business rules - use these six rule types

Every group must write exactly six rules, BR1-BR6. The six rules must have the purposes below. This is to make sure every group has enough behaviour to apply the same DDD ideas.

Rule	Type	What the rule must do	Main concept it should lead to
BR1	Value rule	A rule that defines when a small domain value is valid.	Value Object
BR2	Identity/state rule	A rule about an identified object changing state.	Entity / Aggregate Root
BR3	Invariant rule	A rule that must always hold inside one aggregate, including its child objects or collection.	Aggregate
BR4	Cross-concept rule	A domain decision or calculation that needs information from more than one concept and does not fit naturally on one entity.	Domain Service
BR5	Follow-up rule	After something important happens in Aggregate A, another business action must be requested in Aggregate B.	Domain Event
BR6	Lookup rule	A rule that requires checking or retrieving an existing aggregate before an operation can continue.	Repository + Application Service

For each rule show: the rule statement, the object responsible for enforcing it, and what should happen when the rule is violated.

4. Domain model

Your model must contain the following. Use your six rules to justify each choice.

DDD building block	What is required
Value Object	At least one. It must enforce BR1. Explain why it has no separate identity.
Entities	Use where identity matters. Explain what gives the object its identity.
Two Aggregates	Each must have an Aggregate Root. State the invariant protected by each root.
Domain Service	Use it for BR4. Keep only domain behaviour that does not belong naturally to one entity/value object.
Domain Event	Use it for BR5 to request the follow-up action in the second aggregate.
Repository	One repository abstraction per aggregate root used in the workflow. Use an in-memory implementation.
Factory	Optional. Use only if creation itself has enough rules to justify it.

Do not add a pattern because it appeared in class. If it is optional, explain why you used it or why you left it out.

5. Clean Architecture

Organise the solution into four areas: Domain, Application, Infrastructure and Interface. Your architecture diagram must match the actual source-code dependencies.

Area	What belongs there
Domain	Entities, Value Objects, Aggregate Roots, Domain Services, Domain Events and business rules.
Application	Application Services/use cases, repository abstractions used by the use case, and DTOs.
Infrastructure	In-memory repository implementations and other technical implementations.
Interface	The entry point used to call the application. Keep it minimal.

Show one Application Service for the main use case. It should coordinate the work, not contain the core business rules. Show an input DTO and an output DTO. Demonstrate dependency injection by supplying repository implementations to the Application Service from outside it.

Explain Layer Supertype and state whether you used it. You are not required to implement it.

6. Aggregate communication

Your two use cases must be connected by one Domain Event. Show this flow:

Request -> Application Service -> Aggregate A -> Domain Event -> Handler -> Aggregate B

State the event name, what happened before it was raised, what the handler does, and which rule Aggregate B checks before accepting the follow-up action. Use simple in-process handling.

7. Tests

Provide eight automated tests and use these identifiers in the test names or comments:

- T1-T6: one test for each of BR1-BR6. Across these tests include at least two rejection cases and one boundary case.
- T7: successful main use case. Verify the Domain Event is handled and Aggregate B changes as expected.
- T8: Aggregate B rejects the follow-up action. Verify the final state and returned outcome.

For one test, show the TDD cycle: failing test -> implementation -> passing test. Keep the actual test-run output as evidence.

8. Required slide deck

Submit 15 slides in the order below. Do not add a separate report.

Slide	Required content
1	Title, domain, five members and their roles.
2	Problem, scope, exclusions and key domain terms.
3	BR1-BR6. For each rule: responsible component and violation outcome.
4	Value Object and Entity choices. Link them to BR1 and BR2.
5	Aggregate A and Aggregate B: roots, contents and invariants. Link to BR3.
6	Domain Service for BR4. Factory decision.
7	Repositories: abstractions, in-memory implementations and what each repository stores.
8	Clean Architecture diagram showing Domain, Application, Infrastructure and Interface dependencies.
9	Main Application Service, input/output DTOs and dependency injection.
10	BR5 Domain Event: Aggregate A -> event -> handler -> Aggregate B.
11	T1-T8: what each test checks and expected result.
12	One TDD example: failing test, implementation step, passing test.
13	Walk through the main use case from input DTO to final result.
14	One design choice you rejected or changed, and why.
15	BR1-BR6 traceability: rule -> responsible code/design -> test -> result. Final conclusion.

Keep code off the slides unless a very small excerpt is needed to explain a decision. The presentation is about the design and behaviour, not reading source code.

9. Group roles

Role	Main responsibility
Member 1 - Domain Lead	Scope, terminology and BR1-BR6. Leads Slides 1-3.
Member 2 - Domain Model Lead	Value Objects, Entities, Aggregates, Domain Service and Factory decision. Leads Slides 4-6.
Member 3 - Architecture Lead	Repositories, layers, Application Service, DTOs and dependency injection. Leads Slides 7-9.
Member 4 - Testing Lead	T1-T8, TDD evidence and test results. Leads Slides 11-12.
Member 5 - Integration Lead	Domain Event flow, complete use-case walkthrough, design review and final traceability. Leads Slides 10 and 13-15.

Roles divide preparation work, not understanding. Any member may be asked about any part of the solution.

10. Supporting files

- Final source code for the two use cases.
- The eight automated tests.
- Actual test-run output.
- The failing and passing evidence for the selected TDD example.
- A short README containing the command used to run the tests.

If AI was used, add one short line in the README naming the tool and what it helped with. No prompt log or AI transcript is required.

11. Marking rubric

The coursework has 60 group marks and 40 individual marks.

Group criterion	Marks	Full-credit evidence	Main reasons marks are lost
G1. Domain and rules	10	All six rules follow the required rule types; scope is clear; each rule has a responsible component and violation outcome.	Missing/vague rules, wrong rule type, or no clear responsibility.
G2. DDD model	10	Required DDD building blocks are used correctly and clearly linked to BR1-BR5. Aggregate boundaries and invariants are explained.	Pattern names without a reason, weak boundaries, or business rules placed in the wrong object.
G3. Clean Architecture	10	Actual dependencies match the diagram. Application Service, DTOs, repositories and injection have clear responsibilities.	Layers exist only as folders, dependencies point the wrong way, or domain logic sits in application/infrastructure code.
G4. Domain Event flow	10	BR5 is implemented as a clear event flow from Aggregate A to Aggregate B, with the second aggregate enforcing its own rule.	Event is only a diagram/log message, aggregates directly modify each other, or follow-up behaviour is unclear.
G5. Tests and TDD	10	T1-T8 exist, map to the required behaviour, include rejection/boundary cases, and one genuine fail-then-pass TDD cycle is shown.	Tests are missing, do not map to rules, or only passing screenshots are shown.
G6. Traceability and consistency	10	Slides, business rules, model, source and tests tell the same story. Slide 15 traces every rule to implementation and evidence.	Claims on slides cannot be found in the implementation/tests or the parts contradict each other.

Individual criterion	Marks	What earns full marks
I1. Concept knowledge	10	Explains a requested concept correctly using the group domain, not only a memorised definition.
I2. Design reasoning	10	Explains why a rule belongs in a particular Value Object, Entity, Aggregate, Domain Service or layer.
I3. Trace a use case	10	Can follow one rule from request to application service, domain behaviour, repository/event and test.
I4. Change a requirement	10	Given a small rule change, identifies what should change in the model and which test should be added/changed.

12. Mark limits

- Missing required rule types, pattern-to-rule justification, matching architecture evidence, or the implemented Domain Event limits the related group criterion to 4/10.
- If T1-T8 are incomplete, G5 cannot exceed 7/10. If fail-then-pass TDD evidence is missing, G5 cannot exceed 4/10.
- Unsupported screenshots or claims are not evidence. Test results must come from the submitted project.
- Individual marks come only from what each student can explain and defend.

13. Before you submit

Check that you have exactly six rules, two aggregates, two connected use cases, one Domain Event, eight tests and 15 slides. Check that the same names and rules are used in the slides, source code and tests. Run the tests one final time and keep the output.

The assessment is about whether you can explain how business rules led to your domain model, architecture and tests. Keep the solution small and make every design choice easy to defend.